

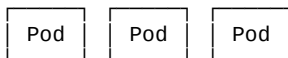
KUBERNETES WORKLOADS

RESOURCE MANAGEMENT

1. ReplicaSets

ReplicaSet ensures a specified number of pod replicas are running at any given time. It replaces failed pods and maintains count. Rarely used directly – Deployments manage ReplicaSets.

ReplicaSet Concept:
Desired replicas: 3



If one pod dies → ReplicaSet creates a new one.

```
# replicaset.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
```

2. Deployments (Manage Rolling Updates, Scaling, Rollbacks)

Deployment manages ReplicaSets and provides declarative updates for pods. Supports rolling updates, rollbacks, and scaling.

2.1 Basic Deployment

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
          ports:
            - containerPort: 80
```

2.2 Scaling the Application

```
# Scale manually
kubectl scale deployment nginx-deploy --replicas=5

# Set autoscaling (HPA) based on CPU
kubectl autoscale deployment nginx-deploy --min=2 --max=10 --cpu-percent=80

# Edit deployment directly
kubectl edit deployment nginx-deploy # change replicas field
```

2.3 Rolling Updates (Zero Downtime)

Deployment updates pods incrementally with configurable strategy.

```
# Update image version (triggers rolling update)
kubectl set image deployment/nginx-deploy nginx=nginx:1.22

# Check rollout status
kubectl rollout status deployment/nginx-deploy

# Rolling update strategy (declarative)
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25% # extra pods allowed during update
      maxUnavailable: 25% # pods that can be down during update
```

2.4 Rollbacks

```
# View rollout history
kubectl rollout history deployment/nginx-deploy

# Rollback to previous revision
kubectl rollout undo deployment/nginx-deploy

# Rollback to specific revision
kubectl rollout undo deployment/nginx-deploy --to-revision=2
```

2.5 Pause and Resume Deployments

```
kubectl rollout pause deployment/nginx-deploy
kubectl rollout resume deployment/nginx-deploy
```

Scenarios:

- Rolling update: Change `image` from nginx:1.21 → 1.22; pods update gradually.
- Rollback after bad release: `kubectl rollout undo` reverts to last good version.
- Scale up before peak load: `kubectl scale --replicas=10`.

3. StatefulSets (Stateful Applications)

StatefulSet manages stateful applications that require stable network identity, persistent storage, and ordered deployment/scaling. Ideal for databases (MySQL, MongoDB, Cassandra).

StatefulSet Characteristics:

```
Stable hostname: pod-name-0, -1, -2
Stable storage: each pod has its own PVC
Ordered deployment (0 → 1 → 2)
Ordered termination (2 → 1 → 0)
```

```
# statefulset.yaml (example with headless service)
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: mysql-h
```

```
  labels:
```

```
    app: mysql
```

```
spec:
```

```
  clusterIP: None # Headless service
```

```
  selector:
```

```
    app: mysql
```

```
  ports:
```

```
  - port: 3306
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: StatefulSet
```

```
metadata:
```

```
  name: mysql
```

```
spec:
```

```
  serviceName: mysql-h
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: mysql
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: mysql
```

```
    spec:
```

```
      containers:
```

```
      - name: mysql
```

```
        image: mysql:8
```

```
        env:
```

```
        - name: MYSQL_ROOT_PASSWORD
```

```
          value: "my-secret-pw"
```

```
        ports:
```

```
        - containerPort: 3306
```

```
        volumeMounts:
```

```
        - name: data
```

```
          mountPath: /var/lib/mysql
```

```
  volumeClaimTemplates:
```

```
  - metadata:
```

```
    name: data
```

```
    spec:
```

```
      accessModes: [ "ReadWriteOnce" ]
```

```
      resources:
```

```
        requests:
```

```
          storage: 10Gi
```

3.1 Operations on StatefulSet

```
# Scale (adds pod-3, then updates storage)
kubectl scale statefulset mysql --replicas=4

# Update image (rolling update)
kubectl set image statefulset/mysql mysql=mysql:8.1

# Delete statefulset (pods, but PVCs survive)
kubectl delete statefulset mysql --cascade=foreground
```

4. DaemonSets (Run Pod on Every Node)

DaemonSet ensures a copy of a pod runs on all (or a subset) nodes. Used for logging agents (Fluentd), monitoring (Prometheus Node Exporter), networking (CNI).

```
DaemonSet:
Node1 → pod (fluentd)
Node2 → pod (fluentd)
Node3 → pod (fluentd)
New node added → DaemonSet automatically schedules pod on it.
```

```
# daemonset.yaml (fluentd example)
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
spec:
  selector:
    matchLabels:
      name: fluentd
  template:
    metadata:
      labels:
        name: fluentd
    spec:
      containers:
        - name: fluentd
          image: fluent/fluentd
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

```
# Commands
kubectl get daemonset
kubectl edit daemonset fluentd
kubectl delete daemonset fluentd
```

5. Jobs & CronJobs (Batch Processing)

5.1 Jobs (One-time Tasks)

Job creates one or more pods and ensures they run to completion. Used for backups, migrations, and batch processing.

```
# job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-job
spec:
  completions: 5      # run 5 pods (sequential/parallel)
  parallelism: 2      # run 2 pods in parallel
  backoffLimit: 4     # retry count
  template:
    spec:
      containers:
        - name: pi
          image: perl
          command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
          restartPolicy: Never
```

```
# Commands
kubectl create job myjob --image=busybox -- echo "hello"
kubectl get jobs
kubectl logs job/pi-job
```

5.2 CronJobs (Scheduled Tasks)

CronJob schedules Jobs at specified times (cron syntax). Use cases: periodic backups, report generation, cleanups.

```
# cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup
spec:
  schedule: "0 2 * * *" # daily at 2am
  startingDeadlineSeconds: 100
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: alpine
              args:
                - /bin/sh
                - -c
                - "echo 'running backup'"
          restartPolicy: OnFailure
```

```
# Commands
kubectl create cronjob my-cron --image=busybox --schedule="*/5 * * * *" -- echo "hello"
kubectl get cronjob
kubectl get jobs --watch
kubectl delete cronjob backup
```

6. Workload Comparison

Resource	Primary Purpose	Scaling	Updating	Use Cases
ReplicaSet	Pod count maintenance	Manual replicas	Rarely updated	Low-level, usually managed by Deployment
Deployment	Stateless apps	Manual/HPA	Rolling updates/rollback	Web apps, APIs, microservices
StatefulSet	Stateful apps	Manual	Ordered rolling update	Databases, message brokers (Kafka)
DaemonSet	Per-node services	N/A (one per node)	Rolling update	Logging, monitoring, node agents
Job	One-time batch	Parallelism/completions	Replace job with new spec	Migration, backup, ETL
CronJob	Periodic schedules	Managed by cron	Edit cronjob spec	Scheduled backups, report generation

Best Practices:

- Use Deployments for stateless apps; never use ReplicaSets alone.
- For databases, prefer StatefulSets + PersistentVolumeClaims.
- DaemonSets are ideal for cluster-wide agents; use nodeSelector to target specific nodes.
- For one-time jobs, use Jobs; for periodic tasks, use CronJobs.
